



Python Programming

Ch-2 Data Structures in Python

Course Outcome : Develop Python program using lists, tuples and dictionaries.

Prof. Perna S. Patil
Department of Computer Engineering
School of Polytechnic & Skill Development

Points to be Covered

Data Structures in Python

2.1 LISTS

Creating Lists, Basic List Operations, Indexing and Slicing in Lists, Built-In Functions Used on Lists, List Methods,

2.2 Tuples: Creating Tuples, Basic Tuple Operations, Indexing and Slicing in Tuples, Built-In Functions Used on Tuples, Relation between Tuples and Lists

2.3 Sets: Creating Set, Basic Set Operations, Built-In Functions Used on Set,

2.4 Dictionaries: Creating Dictionary, Accessing and Modifying key: value Pairs in Dictionaries, Built-In Functions Used on Dictionaries, Dictionary Methods.

Data Structures in Python

INTRODUCTION



Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate and systematic manner/way.

There are four data structures in Python namely, list, tuple, dictionary and set.

- A data structure that stores an ordered collection of items in Python is called a list. In other words, a list holds a sequence of items or elements.

Similar to a list, the tuple is an ordered sequence of items.

A set is an unordered collection of unique items in Python. A dictionary in Python is an unordered collection of key value pairs.

- The data types that are most used in Python are strings, tuples, lists and dictionaries. These are collectively called data structures.

Data Structures in Python

LISTS

Creating Lists,
Basic List Operations,
Indexing and Slicing in Lists,
Built-In Functions Used on Lists,
List Methods,

Data Structures in Python

LISTS



SINCE 1983

Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



A list in Python is a linear data structure.

The elements in the list are stored in a linear order one after other.

A list is a collection of items or elements; the sequence of data in a list is ordered.

The elements or items in a list can be accessed by their positions i.e. indices.

The index in lists always starts with 0 and ends with n-1, if the list contains n elements.

Defining List:

In Python lists are written with square brackets. A list is created by placing all the items (elements) inside a square brackets [], separated by commas.

Syntax for define lists is: `<list_name> = [value1, value2, ... valueN]`

Here, list_name is the name of the list and value1, value2, valueN are list of values assigned to the list.

Example: `Emp = [20, "Amar", 'M', 50, 40]`

Data Structures in Python

Creating a List

The simplest method to create a list by simply assigning a set of values to the list using the assignment operator (=).

We can create a list by placing a comma separated sequence of values in square brackets[].

Example: Creating an empty list.

```
>>> l1=[]          # Empty list
>>> l1              # display l1
[]
>>> l1=list()
>>> l1
[]
```

Data Structures in Python

Creating a list with any integer elements.

```
>>> l2=[10,20,30]      # List of Integers
```

```
>>> l2
```

```
[10, 20, 30]
```

```
>>> l2=list([10,20,30])
```

```
>>> l2
```

```
[10, 20, 30]
```

Example: Creating a list with string elements.

```
>>> l3=["Mango","Orange","Banana"] # List of Strings
```

```
>>> l3
```

```
['Mango', 'Orange', 'Banana']
```

```
>>> l3=list(["red","yellow","green"] )
```

```
>>> l3
```

```
['red', 'yellow', 'green']
```

Data Structures in Python

Accessing Values in List

Accessing elements from a list in Python is a method to get values that are stored in the list at a particular location or index.

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index.

Example: accessing list values.

```
>>> list1 = ["one","two",3,10,"six",20]
```

```
list1[0]='one',list1[1]='two',list1[3]=3
```

```
>>> list1[0]
```

positive indexing

```
'one'
```

```
>>> list1[-2]
```

negative indexing

```
'six'
```

```
>>> list1[1:3]
```

get element from mth index to n-1 index

```
['two', 3]
```

```
>>> list1[3:]
```

get element from mth index to last index

```
[10, 'six', 20]
```

```
>>> list1[:4]
```

get element from 0th index to n-1 index

```
['one', 'two', 3, 10]
```

```
>>>
```

Data Structures in Python

Deleting Values in List

· Python provides many ways in which the elements in a list can be deleted.

1. The pop() method in Python is used to remove a particular item/element from the given index in the list. The pop() method removes and returns the last item if index is not provided. This helps us implement lists as stacks (first in, last out data structure).

Example: for pop() method.

```
>>> fruits = ['apple', 'banana', 'cherry']
```

```
fruits.pop()
```

```
print(fruits)
```

```
o/p: ['apple', 'cherry']
```

Data Structures in Python

2. We can delete one or more items from a list using the keyword `del`. It can even delete the list entirely. But it does not store the value for further use.

Example: Using `del`.

```
>>> list= [10, 20, 30, 40]
>>>list
>>>[10, 20, 30, 40]
>>> del (list[1]) # del() with index
>>> list
[10, 30, 40]
```

The `remove()` method in Python issued to remove a particular element from the list. We use the `remove()` method if we know the item that we want to remove or delete from the list (but not the index).

Example: For `remove()` method.

```
>>> list=[10,"one",20,"two"] # heterogeneous list
>>> list.remove(20) # remove element 20
>>> list
[10, 'one', 'two']
>>> list.remove("one") # remove element one
>>> list
[10, 'two']
```

Data Structures in Python

Updating Lists (Change or Add Elements to a List)

- Lists are mutable, meaning their elements can be changed or updated unlike strings or tuples.
- Multiple values can be added into a list. We can use the assignment operator (=) to change an item or a range of items.
- We can update items of the list by simply assigning the value at the particular index position. We can also remove the items from the list using `remove()` or `pop()` or `del` statement.

Example: For updating lists.

```
>>> list1= [10, 20, 30, 40, 50]
>>> list1
[10, 20, 30, 40, 50]
>>> list1[0]=0                # change 0th index element
>>> list1
[0, 20, 30, 40, 50]
>>> list1[-1]=60              # change last index element
>>> list1
[0, 20, 30, 40, 60]
>>> list1[1]=[5,10]           # change 1st index element as sublist
>>> list1
[0, [5, 10], 30, 40, 60]
>>> list1[1:1]=[3,4]           # add elements to a list at the desired location
```

Data Structures in Python

Following table shows the list methods used for updating list.

`append()` `list.append(item)`

`extend()` `list1.extend(list2)`

`insert()` `list.insert(index, element)`

For `append()` method.

```
>>> list1=[10,20,30]
>>> list1
[10, 20, 30]
>>> list1.append(40) # add
element at the end of list
>>> list1
[10, 20, 30, 40]
```

Program for `extend()` method.

```
>>>list1=[10, 20, 30, 40]
>>>list1
[10, 20, 30, 40]
>>> list1.extend([60,70])
#add elements at the end
of list
>>> list1
[10, 20, 30, 40, 60, 70]
```

Program for `insert()` method.

```
>>> list1=[10, 20]
>>>list1
[10,20]
>>> list1.insert(1,30)
>>> list1
[10, 30, 20]
```

Data Structures in Python

Basic List Operations (indexing and slicing)

The operations on list include indexing and slicing.

Indexing

An individual item in the list can be referenced by using an index, which is an integer number that indicates the relative position of the item in the list.

There are various ways in which we can access the elements of a list some as them are given below:

1. List Index: We can use the index operator `[]` to access an item in a list. Index starts from 0. So, a list having 5 elements will have index from 0 to 4.

Negative Indexing: Python allows negative indexing for its sequences. The index of `-1` refers to the last item, `-2` to the second last item and so on.

list2=	p	y	t	h	o	n
	-6	-5	-4	-3	-2	-1

list1[4]	→	50
list1[3]	→	40
list1[2]	→	30
list1[1]	→	20
list1[0]	→	10

List has Five Elements with Index 0 to 4

Data Structures in Python

List Slicing

Slicing is an operation that allows us to extract elements from units. The slicing feature used by Python to obtain a specific subset or element of the data structure using the colon (:) operator. The slicing operator returns a subset of a list called slice by specifying two indices, i.e. start and end.

Syntax:

```
list_variable[start_index:end_index]
```

- This will return the subset of the list starting from start_index to one index less than that of the end index.

For slicing list.

```
>>> l1=([10,20,30,40,50])
```

```
>>> l1[1:4]
```

```
[20, 30, 40]
```

```
>>>l1[2:5]
```

```
[30,40,50]
```

Built-in Functions and Methods for list

Python provide certain method and function work with list. Following table shows different functions with their description and example.



Function	Description	Example
len(list).	It returns the length of the list	>>> list1 [1, 2, 3, 4, 5] >>> len(list1) 5
max(list)	It returns the item that has the maximum value in a list.	>>> list1 [1, 2, 3, 4, 5] >>> max(list1) 5
sum(list)	Calculates sum of all the elements of list.	>>>list1 [1, 2, 3, 4, 5] >>>sum(list1) 15
min(list)	It returns the item that has the minimum value in a list.	
list(seq)	It converts a tuple into a list.	

Methods in Lists:



Dr. Vishwanath Karad

MIT WORLD PEACE
UNIVERSITY | PUNE

ARCH, SOCIAL INNOVATION & PARTNERSHIPS

SINCE 1983



Function	Description	Example
<code>list.append(item)</code>	It adds the item to the end of the list.	<pre>>>> list1 [1, 2, 3, 4, 5] >>> list1.append(6) >>> list1 [1, 2, 3, 4, 5, 6]</pre>
<code>list.count(item)</code>	It returns number of times the item occurs in the list.	
<code>list.extend(seq)</code>	It adds the elements of the sequence at the end of the list.	
<code>list.index(item)</code>	It returns the index number of the item. If item appears more than one time, it returns the lowest index number.	
<code>list.insert(index,item)</code>	It inserts the given item onto the given index number while the elements in the list take one right shift.	

Data Structures in Python

Function	Description	Example
<code>list.pop(item=list[-1])</code>	It deletes and returns the last element of the list.	
<code>list.remove(item)</code>	It deletes the given item from the list.	
<code>list.reverse()</code>	It reverses the position (index number) of the items in the list.	
<code>list.sort()</code>	Sorts items in the list.	
<code>list.sort([func])</code>	It sorts the elements inside the list and uses compare function if provided.	

TUPLES

A tuple is a collection items which is ordered and unchangeable.

A tuple is a data structure that is an immutable or unchangeable, ordered sequence of elements/items. Because tuples are immutable, their values cannot be modified.

A tuple is an immutable data type. An immutable data structure means that we cannot add or remove items from the tuple data structure.

In Python tuples are written with round brackets () and values in tuples can also be accessed by their index values, which are integers starting from 0.

Creating Tuple

To create tuple, all the items or elements are placed inside parentheses () separated by commas and assigned to a variable.

The **syntax** for defining a tuple in Python is: **<tuple_name> = (value1, value2, ... valueN).**

Here help name indicate name as the tuple and value1, value2,...valueN are the values assigned to the tuple.

Example: Emp (20, "Amar", 'M', 50, 40)

A tuple in Python is an immutable data type which means a tuple once created cannot be altered/modified. Tuples can have any number of different data items (integer, float, string, list etc)

Data Structures in Python

For creating tuples.

```
>>> tuple1=(10,20,30) # A tuple with integer values
>>> tuple1
(10, 20, 30)
>>> tuple2=(10,"abc",11.22,'X') # A tuple with different data types
>>> tuple2
```

```
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
```

```
print(tuple1)
print(tuple2)
print(tuple3)
```

Data Structures in Python

Accessing Values in Tuple

Accessing items/element from the tuple is a method to get values stored in the tuple from a particular location or index.

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index.

Example: For accessing values in tuples.

```
>>> tuple=(10,20,30,40,50)
>>> tuple[1]          #access value at specific index
20
>>> tuple[1:4]         #tuple[m:n] will return elements from mth index to n-1th index.
(20, 30, 40)
>>> tuple[:2]          #tuple[:n] will return elements from 0th index to n-1th index.
(10, 20)
>>> tuple[2:]          #tuple[m:] will return elements from mth index to last index.
(30, 40, 50)
>>> tuple[-1]          #access value at last index
50
```

Data Structures in Python

Deleting Tuples

- Tuples are unchangeable, so we cannot remove items from it, but we can delete the tuple completely. To explicitly remove an entire tuple, just use the del statement.

Example: Delete entire tuple.

```
>>> t1
```

```
(10, 20)
```

```
>>> del t1
```

```
>>> t1
```

Traceback (most recent call last):

File "<pyshell#65>", line 1, in <module>

```
t1
```

NameError: name 't1' is not defined

Data Structures in Python

Updating Tuple

· Tuples are immutable which means we cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples.

Example: For updating tuple.

```
>>> tuple1  
(10, 20, 30)  
>>> tuple2  
( 'A', 'B', 'C' )  
>>> tuple1[1]=40 # get error as tuples are immutable  
Traceback (most recent call last):  
File "<pyshell#39>", line 1, in <module>  
tuple1[1]=40  
TypeError: 'tuple' object does not support item assignment  
>>> tuple3=tuple1+tuple2 # concatenating two tuples  
>>> tuple3  
(10, 20, 30, 'A', 'B', 'C')
```

Change value

In order to change a value, we can convert tuple into list and then change the values and revert back list into tuple can solve the purpose of update.

Example:

```
tuple1 = (1, 2, 3, 4, 5)
```

```
list1 = list (tuple1)
```

```
list1[2]=20
```

```
b = tuple (list1)
```

```
print(b)
```

Output:

```
(1, 2, 20, 4, 5)
```

Data Structures in Python

Tuple Operations

- A set of operations can be apply to Python tuple. The operations described below are supported by most sequence types, both mutable and immutable. Here we will consider different operations in context of immutable sequence.

1. Concatenation and Repetition:

- We can use + operator to combine two tuples. This is also called concatenation operation.
- We can also repeat the elements in a tuple for a given number of times using the * operator.

Example: For tuple operations using + and * operators.

```
>>> t1=(10,20)
```

```
>>> t2=(30,40)
```

```
>>> t1+t2
```

```
(10, 20, 30, 40)
```

```
>>> t1*2
```

```
(10, 20, 10, 20)
```

Data Structures in Python

Membership Function:

- We can test if an item exists in a tuple or not, using the keyword in. The in operator evaluates to true if it finds a variable in the specified sequence and false otherwise.

Example: For membership function in tuple.

```
>>> tuple  
(10, 20, 30, 40, 50)  
>>> 30 in tuple  
True  
>>> 25 in tuple  
False
```

Iterating through a Tuple:

- Iteration over a tuple specifies the way which the loop can be applied to the tuple.
- Using a for loop we can iterate through each item in a tuple. Following example uses a for loop to simply iterates over a tuple.

Example: For iterating items in tuple using for loop.

```
>>> tuple=(10,20,30)  
>>> for i in tuple:  
print(i) #use two enter to get the output
```

Output:

```
10  
20  
30
```

Data Structures in Python

Build-in Tuple Functions

Following table built-in tuple function in Python programming.

Function	Description	Example
cmp(tuple1, tuple2)	Compares elements of both tuples.	<pre>>>> tup1=(1,2,3) >>> tup2=(1,2,3) >>> cmp(tup1,tup2) 0</pre>
len(tuple)	Gives the total length of the tuple.	<pre>>>> tup1 (1, 2, 3) >>> len(tup1) 3</pre>
max(tuple) min(tuple) same working	Returns item from the tuple with max value.	<pre>>>> tup1 (1, 2, 3) >>> max(tup1) 3</pre>
index()	Searches the tuple for a specified value and returns the position of where it was found	<pre>> >> tup1 (1, 2, 3) >>> tup1.index(3) 2</pre>
tuple(seq)	Converts a list into tuple.	<pre>>>>tup1 = (1, 2, 3, 4, 5) >>>list1 = list (tup1) >>>list1 [1,2,3,4,5]</pre>

Example tuple

#create tuple

```
my_data = ("hi", "hello", "bye")
```

```
print(my_data)
```

```
print(my_data[0])
```

```
my_data1 = (1, 2, "Kevin", 8.9)
```

#Negative indexes in tuples/

```
print(my_data1[-1])
```

#Accessing elements from nested tuples

```
my_data2 = (1, "Steve", (11, 22, 33))
```

```
print(my_data2[1][3])
```

#Changing the elements of a tuple

```
my_data3 = (1, [9, 8, 7], "World")
```

```
print(my_data3)
```

```
my_data3[1][2] = 99
```

```
print(my_data3)
```

#Slicing operation in tuples

```
my_data4 = (11, 22, 33, 44, 55, 66, 77, 88, 99)
```

```
print(my_data4)
```

```
print(my_data[2:5])
```

```
print(my_data[:4])
```

```
print(my_data[4:])
```

#Membership Test in Tuples

```
my_data5 = (11, 22, 33, 44, 55, 66, 77, 88, 99)
```

```
print(my_data5)
```

```
print(22 in my_data)
```

```
print(88 not in my_data)
```

Dictionary

Creating Dictionary, Accessing and Modifying key: value Pairs in Dictionaries, Built-In Functions Used on Dictionaries, Dictionary Methods.

Dictionary is a mutable data type in Python. A python dictionary is a collection of key and value pairs separated by a colon (:), enclosed in curly braces {}.

Left side of the colon(:) is the key and right side of the : is the value.

Syntax for define a dictionary is as follows

```
<dictionary_name> = {key1:value1, key2:value2...keyN:valueN}
```

Example: Emp = {"ID": 20, "Name" : "Amar", "Gender"=Male, "SalaryPerHr":200}

A pair of curly braces with no values in between is known as an empty dictionary.

Dictionary items are accessed by keys, not by their position (index).

The values in a dictionary can be duplicated, but the keys in the dictionary are unique.

Dictionaries are changeable (mutable).

We can change or update the items in dictionary as and when required

Creating Dictionary:

Creating a dictionary is as simple as placing items inside curly braces {} separated by commas. An item has a key and a corresponding value that is expressed as a pair (**key: value**).

empty dictionary

```
my_dict = {}
```

dictionary with integer keys

```
my_dict = {1: 'apple', 2: 'ball'}
```

dictionary with mixed keys

```
my_dict = {'name': 'John', 1: [2, 4, 3]}
```

Python provides a build-in function dict() for creating a dictionary.

Creating directory using dict()

Accessing Values in a Dictionary

We can access the items of a dictionary by following ways:

1. Keys can be used either inside square brackets **[]** or **with the get() method**.

```
E.g. my_dict = {'name': 'Jack', 'age': 26}  
print(my_dict['name'])  
print(my_dict.get('age'))
```

O/P:

Changing and Adding Dictionary elements



SINCE 1983
Dr. Vishwanath Karad
MIT WORLD PEACE
UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Changing and Adding Dictionary elements

Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

If the key is already present, then the existing value gets updated. In case the key is not present, a new (**key: value**) pair is added to the dictionary.

Changing and adding Dictionary Elements

```
my_dict = {'name': 'Jack', 'age': 26}
```

update value

```
my_dict['age'] = 27
```

```
print(my_dict)
```

```
#Output: {'age': 27, 'name': 'Jack'}
```

add item

```
my_dict['address'] = 'Downtown'
```

```
print(my_dict)
```

```
# Output: {'address': 'Downtown', 'age': 27, 'name': 'Jack'}
```

Deleting Elements/Items From Dictionary

Deleting Elements/Items From Dictionary

We can remove a particular item in a dictionary by using the method `pop()`.

This method removes an item with the provided key and returns the value.

- The method, `popitem()` can be used to remove and return an arbitrary item (key, value) from the dictionary.
- All the items can be removed at once using the `clear()` method.

We can also use the **`del` keyword** to remove individual items or the entire dictionary itself.

Deleting Elements/Items From Dictionary

Removing elements from a dictionary

create a dictionary

```
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

remove a particular item, returns its value

```
print(squares.pop(4))
```

```
print(squares)
```

remove an arbitrary item, return (key,value)

```
print(squares.popitem())
```

```
print(squares)
```

remove all items

```
squares.clear()
```

```
print(squares)
```

delete the dictionary itself

```
del squares
```

Throws Error

```
print(squares)
```

Dictionary Program

Write a Python program to concatenate following dictionaries to create a new one.

```
dic1={1:10, 2:20}
```

```
dic2={3:30, 4:40}
```

```
dic3={5:50,6:60}
```

```
dic4 = {}
```

```
for d in (dic1, dic2, dic3): dic4.update(d)
```

```
print(dic4)
```

Updating Dictionary

Updating Dictionary

- The dictionary data type is flexible data type that can be modified according to the requirements which makes it is a mutable data type.

The dictionary are mutable means changeable. We can add new items or change the value of existing items using assignment operator.

- If the key is already present, value gets updated, else a new key:value pair is added to the dictionary.

```
>>> dict1
{'name': 'vijay', 'age': 40}
>>> dict1['age']=35 # updating values in dictionary
>>> dict1
{'name': 'vijay', 'age': 35}
>>> dict1['address']='thane' # add new item in dictionary
>>> dict1
{'name': 'vijay', 'age': 35, 'address': 'thane'}
```

Basic Operations on Directory

Dictionary Membership Test:

· We can test if a key is in a dictionary or not using the keyword in. Notice that membership test is for keys only, not for values

For directory membership test.

```
>>> squares={1:1,2:4,3:9,4:16,5:25}
```

```
>>> squares
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
>>> print(1 in squares)
```

```
True
```

```
>>> print(6 in squares)
```

```
False
```

```
>>> squares={1:1,2:4,3:9,4:16,5:25}
```

```
>>> print(9 in squares)
```

```
False
```

Traversing Dictionary:

Traversing Dictionary:

- Using a for loop we can iterate through each key in a dictionary.

Example: For traversing dictionary.

```
>>> squares
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
>>> for i in squares:
```

```
    print(i,squares[i])
```

Dictionary Methods and Built-in Functions

· Python has a set of built-in methods that we can use on dictionaries. Sc given in following table.



SINCE 1983
Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Sr. No.	Method	Description	Example
1.	<code>clear()</code>	Removes all the elements from the dictionary.	<pre>dict={1:'Vijay',2:'Amar',3:'Santosh'} >>> dict {1: 'Vijay', 2: 'Amar', 3: 'Santosh'} >>> dict.clear() >>> dict {}</pre>
2.	<code>copy()</code>	Returns a copy of the dictionary.	<pre>>>> dict={1: 'Vijay', 2: 'Amar', 3: 'Santosh'} >>> X=dict.copy() >>> X {1: 'Vijay', 2: 'Amar', 3: 'Santosh'}</pre>
3.	<code>fromkeys()</code>	The <code>fromkeys()</code> method creates a new dictionary with default value for all specified keys. If default value is not specified, all keys are set to <code>None</code> .	<pre>>>> dict=dict.fromkeys(['Vijay','Meenakshi'], 'Author') >>> dict {'Vijay': 'Author', 'Meenakshi': 'Author'} >>></pre>
4.	<code>get()</code>	Returns the value of the specified key.	<pre>>>> dict1={'name':'vijay','age':40} >>> dict1.get('name') 'vijay'</pre>
5.	<code>items()</code>	Returns a list containing the a tuple for each key value pair.	<pre>dict={1: 'Vijay', 2: 'Amar', 3: 'Santosh'} >>> for i in dict.items(): print(i) (1, 'Vijay') (2, 'Amar') (3, 'Santosh')</pre>

Dictionary Methods and Built-in Functions

- Python has a set of built-in methods that we can use on dictionaries. Some of them are given in following table.

			(3, 'Santosh')
6.	keys()	Returns a list containing the dictionary's keys.	<pre>dict={1:'Vijay',2:'Amar',3:'Santosh'} >>> dict.keys() dict_keys([1, 2, 3])</pre>
7.	pop()	Removes the element with the specified key.	<pre>dict={1:'Vijay',2:'Amar',3:'Santosh'} >>> print(dict.pop(2))</pre>

Dictionary Methods and Built-in Functions

· Python has a set of built-in methods that we can use on dictionaries. Some of them are given in following table.

8.	<code>popitem()</code>	Removes the last inserted key-value pair.	<pre>dict={1:'Vijay',2:'Amar',3:'Santosh'} >>> dict.popitem() (3, 'Santosh')</pre>
9.	<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value.	<pre>>>> dict {2: 'Amar', 3: 'Santosh'} >>> dict.setdefault(1,'Vijay') >>> dict {2: 'Amar', 3: 'Santosh', 1: 'Vijay'}</pre>
10.	<code>update()</code>	Updates the dictionary with the specified key-value pairs.	<pre>>>> dict1 {2: 'Amar', 4: 'Umesh'} >>> dict2={1:'Vijay',3:'Santosh'} >>> dict2 {1: 'Vijay', 3: 'Santosh'} >>> dict1.update(dict2) >>> dict1 {2: 'Amar', 4: 'Umesh', 1: 'Vijay', 3: 'Santosh'}</pre>
11.	<code>values()</code>	Returns a list of all the values in the dictionary.	<pre>>>> dict {1: 'Vijay', 2: 'Amar', 3: 'Santosh'} >>> dict.values() dict_values(['Vijay', 'Amar', 'Santosh'])</pre>

Dictionary Methods and Built-in Functions

• Python has a set of built-in methods that we can use on dictionaries. Some of them are given in following table.

Built-in Functions of Dictionary:			
Sr. No.	Function	Description	Example
1.	all()	Return True if all keys of the dictionary are true (or if the dictionary is empty).	<pre>>>> dict {1: 'Vijay', 2: 'Amar', 3: 'Santosh'} >>> all(dict) True</pre>
2.	any()	Return True if any key of the dictionary is true. If the dictionary is empty, return False.	<pre>>>> dict={} >>> any(dict) False</pre>
3.	len()	Return the length (the number of items) in the dictionary.	<pre>>>> dict {1: 'Vijay', 2: 'Amar', 3: 'Santosh'} >>> len(dict) 3</pre>
4.	sorted()	Return a new sorted list of keys in the dictionary.	<pre>>>> dict1 {2: 'Amar', 1: 'Vijay', 4: 'Umesh', 3: 'Amar'} >>> sorted(dict1) [1, 2, 3, 4]</pre>
5.	cmp()	Compares two dictionaries based on key and values.	<pre>>>> dict1 {2: 'Amar', 1: 'Vijay', 4: 'Umesh', 3: 'Vikas'}</pre>

Python Sets



The set data structure in Python programming is implemented to support mathematical set operations.

Set is an unordered collection of unique items. Items stored in a set are not kept in any particular order.

A set data structure in python programming includes an unordered collection of items without duplicates.

Sets are unindexed that means we cannot access set items by referring to an index.

Sets are changeable (mutable) i.e., we can be changed or update a set as and when required.

Type of elements in a set need not be the same, various mixed up data type values can also be passed to the set.

- The sets in python are typically used for mathematical operations like union, intersection, difference and complement etc.

Python Sets

Accessing Values in Sets

A set is used to contain an unordered collection of items. There are two ways for creation of sets in python:

1. Set is defined by values separated by comma inside braces { }. Items in a set are not ordered.
2. **Syntax** for defining set is: <set_name>={value1, value2,....,valueN}

Example: For creating sets with { }.

```
>>> a={1,3,5,4,2}
>>> print("a=",a)
a= {1, 2, 3, 4, 5}
>>> print(type(a))
<class 'set'>
>>> colors = {'red', 'green', 'blue','red'}
>>> colors
{'blue', 'red', 'green'}
>>>
```

- A set does not contain any duplicate values or elements. We can perform set operations like union, intersection on two sets. Set have unique values. They eliminate duplicates.

Python Sets

Deleting Values in Set

· There are different ways for deletion of sets in Python programming. Some of them are given below:

1. Remove elements from a set by using discard() method.
2. Remove elements from a set by using remove() method. The only difference between remove and discard method is that If specified item is not present in a set then remove() method raises KeyError.
3. pop() method removes random item from a set and returns it.
4. clear() method remove all items from the set.

Example: For deleting values in sets.

Example: For deleting values in sets.

For deleting values in sets.

```
>>> b={"a","b","c"}
```

```
>>> b
```

```
{'c', 'b', 'a'}
```

```
>>> b.discard("b")
```

```
>>> b
```

```
{'c', 'a'}
```

```
>>> b.remove("a")
```

```
>>> b
```

```
{'c'}
```

```
>>> b={"a","b","c"}
```

```
>>> b.pop()
```

```
'c'
```

```
>>> b.clear()
```

```
>>> b
```

```
set()
```

Updating Set

- We can update a set by using add() method or update() method.

1. Using add() Method:

- We can add elements to a set by using add() method. The () method takes one argument which is the element we want to add in set.
- At a time only one element can be added to the set by using add() method, loops are used to add multiple elements at a time with the use of add() method.

Syntax: A.add(element)

Here, A is set which will be updated by given element.

Example: For add() method which updates sets.

```
>>> a={1,2,3,4}
>>> a.add(6)
>>> print(a)
{1, 2, 3, 4, 6}
>>> a={2,4,6,8}
>>> a.add(3)
>>> print(a)
{2, 3, 4, 6, 8}
>>>
```

Using update() Method:

- The update() adds elements from lists/strings/tuples/sets to the set.
- The update() method can accept lists, strings, tuples as well as other sets as its arguments. In all of these cases, duplicate elements are avoided.

Syntax: A.update(B)

Here, B can be lists/strings/tuples/sets and A is set which will be updated without duplicate entries.

Example: For update() method which updates sets.

```
>>> a={1,2,3}
>>> b={'a','b','c'}
>>> a
{1, 2, 3}
>>> b
{'c', 'b', 'a'}
>>> a.update(b)
>>> a
{1, 2, 3, 'b', 'a', 'c'}
>>> a.update({3,4})
>>> a
{1, 2, 3, 'b', 4, 'a', 'c'}
```

Basic Set Operations

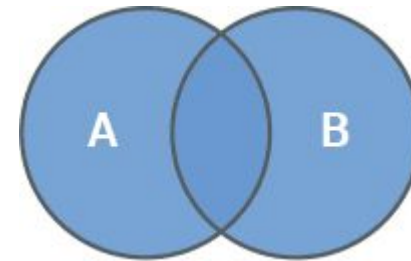
Union, Intersection, Difference and Symmetric Difference are some operations which are performed on sets.

1. Union:

- Union operation performed on two sets returns all the elements from both the sets.
- The union of A and B is defined as the set that consists of all elements belonging to either set A or set B (or both). It is performed by using $|$ operator.

Example: For union operation in sets.

```
>>> A={1,2,4,6,8}
>>> B={1,2,3,4,5}
>>> C=A | B
>>> C
{1, 2, 3, 4, 5, 6, 8}
```



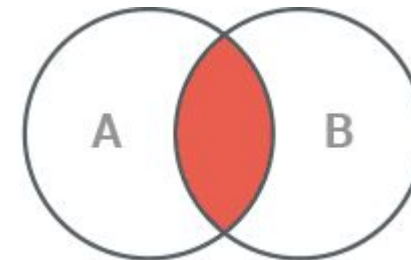
Union
Operation

Intersection:

- Intersection operation performed on two sets returns all the elements which are common or in both the sets.
- The intersection of A and B is defined as the set composed of all elements that belong to both A and B. It is performed by using $&$ operator.

Example: For intersection operation in sets.

```
>>> A={1,2,4,6,8}
>>> B={1,2,3,4,5}
>>> C=A & B    >>> C    {1, 2, 4}
```



Intersection
operation

Basic Set Operation

Difference:

· Difference operation on two sets set1 and set2 returns all the elements which are present on set1 but not in set2. It is performed by using – operator.

Example: For difference operation in sets.

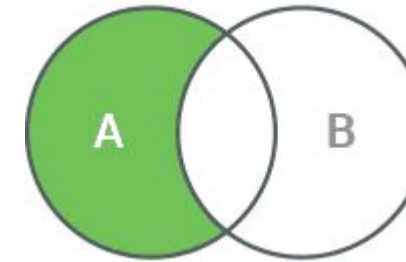
```
>>> A={1,2,4,6,8}
```

```
>>> B={1,2,3,4,5}
```

```
>>> C=A - B
```

```
>>> C
```

```
{8, 6}
```



**Difference
Operation**

Symmetric Difference:

· Symmetric Difference operation performed by using ^ operation.

Example: For symmetric difference operation in sets.

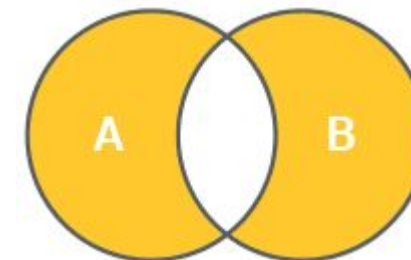
```
>>> A={1,2,4,6,8}
```

```
>>> B={1,2,3,4,5}
```

```
>>> C=A ^ B
```

```
>>> C
```

```
{3, 5, 6, 8}
```



**Symmetric
Difference
Operation**

Built-in Set Functions and Methods

Methods in set:

Method	Description	Syntax
<code>union()</code>	Return a new set containing the union of two or more sets	<code>set.union(set1, set2...)</code>
<code>intersection()</code>	Returns a new set which is the intersection of two or more sets	<code>set.intersection(set1, set2...)</code>
<code>intersection_update()</code>	Removes the items from this set that are not present in other sets	<code>set.intersection_update(set1, set2...)</code>
<code>difference()</code>	Returns a new set containing the difference between two or more sets	<code>set.difference(set1, set2...)</code>
<code>difference_update()</code>	Removes the items from this set that are also included in another set	<code>set.difference_update(set1, set2...)</code>
<code>symmetric_difference()</code>	Returns a new set with the symmetric	<code>set.symmetric_difference(set)</code>

Built-in Set Functions and Methods



SINCE 1983
Dr. Vishwanath Karad
MIT WORLD PEACE
TY | PUNE
INNOVATION & PARTNERSHIPS

Supported by
UNESCO
UNESCO Chair for
Human Rights, Democracy, Peace & Tolerance
World Peace Centre (Mumbai) Pune, India

	differences of two or more sets	
<code>symmetric_difference_update()</code>	Modify this set with the symmetric difference of this set and other set	<code>set.symmetric_difference_update(set)</code>
<code>isdisjoint()</code>	Determines whether or not two sets have any elements in common	<code>set.isdisjoint(set)</code>
<code>issubset()</code>	Determines whether one set is a subset of the other	<code>set.issubset(set)</code>
<code>issuperset()</code>	Determines whether one set is a superset of the other	<code>set.issuperset(set)</code>
<code>add(item)</code>	It adds an item to the set. It has no effect if the item is already present in the set.	<code>set.add(set)</code>
<code>discard(item)</code>	It removes the specified item from the set.	<code>set.discard(set)</code>
<code>remove(item)</code>	Remove an element from a set; it must be a member. If the element is not a member, raise a <code>KeyError</code> .	<code>set.remove(set)</code>
<code>pop()</code>	Remove and return an arbitrary set element that is the last element of the set. Raises <code>KeyError</code> if the set is empty.	<code>set.pop(set)</code>
<code>update()</code>	Updates the set with the union of itself and others.	<code>set.update(set)</code>

built-in function of Set



- Built-in functions like `all()`, `any()`, `enumerate()`, `len()`, `max()`, `min()`, `sorted()`, `sum()` etc. are commonly used with set to perform different tasks.

Following table lists built-in function of Set

Sr. No.	Function	Description
1.	<code>all()</code>	Return True if all elements of the set are true (or if the set is empty).
2.	<code>any()</code>	Return True if any element of the set is true. If the set is empty, return False.
3.	<code>enumerate()</code>	Return an enumerate object. It contains the index and value of all the items of set as a pair.
4.	<code>len()</code>	Return the length (the number of items) in the set.
5.	<code>max()</code>	Return the largest item in the set.
6.	<code>min()</code>	Return the smallest item in the set.
7.	<code>sorted()</code>	Return a new sorted list from elements in the set(does not sort the set itself).
8.	<code>sum()</code>	Retrun the sum of all elements in the set.

Set Example

Example: For set functions.

```
A = {'red', 'green', 'blue'}
B = {'yellow', 'red', 'orange'}
print("Union ", A.union(B))
print("Intersection " ,A.intersection(B))
A.intersection_update(B)
print("Intersection Update ", A)
A = {'red', 'green', 'blue'}
print("Difference ",A.difference(B))
A.difference_update(B)
print("Difference Update ",A)
A = {'red', 'green', 'blue'}
print("Symmetric Difference", A.symmetric_difference(B))
A = {'red', 'green', 'blue'}
A.symmetric_difference_update(B)
print("Symmetric Difference Update",A)
A={'red'}
print(A.isdisjoint(B))
print(A.issubset(B))
print(A.issuperset(B))
```

Output:

```
Union {'yellow', 'orange', 'green', 'red', 'blue'}
Intersection {'red'}
Intersection Update {'red'}
Difference {'green', 'blue'}
Difference Update {'blue', 'green'}
Symmetric Difference {'orange', 'blue', 'yellow', 'green'}
Symmetric Difference Update {'blue', 'orange', 'yellow', 'green'}
False
True
False
```

Python Operators



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Python Operators



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Python Operators



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

